

Test and Reliability Exercise Sheet 2

Exercise 1

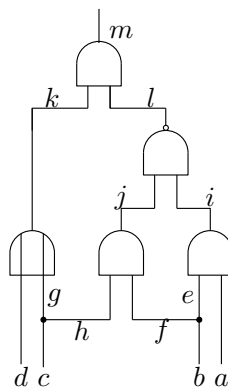


Figure 1: Circuit for Exercise a)

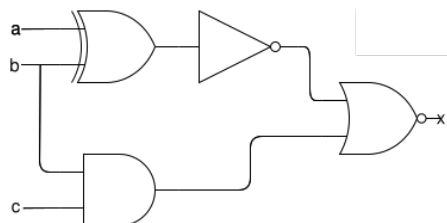


Figure 2: Circuit for Exercise b)

- a) Execute a fault parallel simulation for the circuit in figure 1 for the following faults:

fault-free circuit, s-a-1 fault at j , s-a-0 fault at l , s-a-1 fault at k

The pattern to be simulated is $(a, b, c, d) = (0101)$.

Give the simulated assignment for each signal $a, \dots, m$ in the circuit. Which of the above faults are discovered by this test pattern?

- b) Perform Equivalence Fault Collapsing (EFC) and Dominance Fault Collapsing (DFC) separately to the circuit of figure 2. How many stuck-at faults can occur in this circuit with EFC, DFC and without fault collapsing?

Exercise 2

Consider the circuit in Figure 3. In total the circuit has 22 lines, thus the total number of possible stuck-at faults is 44. Perform a deductive fault simulation of the test patterns (1001) and (1111) and give the list of detected faults for every line. Please consider that fault effects may reconverge. What can be said about the quality of the two test patterns ?

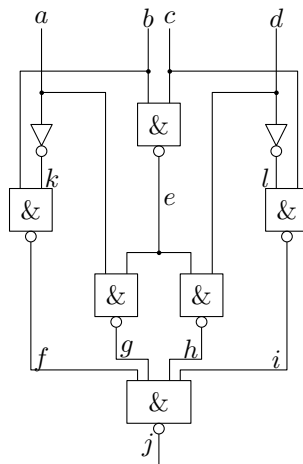


Figure 3: Circuit

Exercise 3

Consider the following C code snippet from an implementation for *parallel* simulation of circuit models represented as And-Inverter-Graphs (AIGs). AIGs are representations of Boolean functions which only consist of AND and inverter gates (by DeMorgan's law, every arbitrary digital combinational circuit can be transformed into an AIG).

Assume variables that represent circuit lines are encoded as follows (very common so called 'odd-even' encoding):

Given a variable v , if the respective line is unnegated we write $2 \cdot v$, if it is negated, we denote it by $2 \cdot v + 1$. Thus, odd numbers represent negated (by an inverter) variables. We call these numbers *literals* of variables. Technically, the sign of the literal is represented by the least significant bit position. (In the spirit of the AIGER standard: <https://fmv.jku.at/aiger/>)

An example:

12 3 8

Specifies that the AND-gate which computes variable 6 (literal 12) has inputs 1 and 4. Whereas 1 is negated (negated literal 3, odd) and 4 not (literal 8, even). The output of an AND-gate specification is never negated (only if it is again used as an input in another specification).

Another example, XOR-gate as AIG (comments are introduced via #):

```

2          #input1
4          #input2
6 2 5      #6 = and(input1, not(input2))
8 3 4      #8 = and(not(input1), input2)
10 7 9
11         #output (negated because or(a,b) = not(and(not(a),not(b)))

```

Further assume that the current assignment of values to variables (lines in the AIG circuit) are stored in an array called `current_valuation`. Inputs are already assigned (you do not have to consider their initialization).

We do bit-parallel simulation and exploit the bitwidth of an unsigned int word.

```

...
unsigned int deref(unsigned int literal)
{
    unsigned int variable_of_literal = literal / 2;
    unsigned int result = current_valuation[variable_of_literal]
    unsigned int negated_literal = literal & 1;
    if (negated_literal)
    {
        ...
        // something is missing here, Hint: UINT_MAX = 11...111
        ...
    }
    return result;
}
...
unsigned int l, r, tmp, j;
for (j = 0; j < model->num_and; j++) // iterate over all gates
{
    and_gate *and = model->ands + j;
    l = deref (and->rhs0); // and->rhs0 is the first input literal of AND
    r = deref (and->rhs1); // and->rhs1 is the second input literal of AND
    ...
    // something is missing here
    ...
    current_valuation[and->lhs / 2] = ...; // store output value (and->lhs)
}

```

- Explain the given C code parts. How is it possible to simulate with real parallelism on a general purpose CPU?
- Complete the part denoted by 'something is missing here', such that the `current_valuation[and->lhs / 2]` fields actually will hold the correct output of each AND gate.

Deadline: Discussion on Thursday, May 21, 2026